FitPick

Digital Wardrobe & Outfit Planner

Project Report

Software Engineering · Final Project Submission (Part C) · Revised

Author **Nithilan A (nithi0307)**
Repository **github.com/nithi0307/fitpick**
Stack **React 19 + Vite · FastAPI · PostgreSQL 16 · Docker**

Revision note

This document has been revised and re-uploaded at the request of the course instructor. After the original submission the application was extended — per-account wardrobes, a ready-made clothing catalogue, removal of items, and saved outfits with a style score — and this report has been updated throughout so that it describes the software as it now stands. Every figure, screenshot and test result below was re-captured from the current build.

Contents

1. Problem statement
2. User stories
3. System architecture and system design
4. Design of tests
5. Appendix A — Software Requirements Specification
6. Appendix B — Data Flow Diagrams
7. Appendix C — Entity Relationship Diagram
8. Appendix D — UML diagrams
9. Appendix E — Code listing and GitHub link

1. Problem statement

1.1 The problem

Most people own far more clothes than they wear. Items get pushed to the back of a wardrobe and forgotten, near-duplicates are bought because nobody can remember what is already owned, and the outfit that worked well last month is never written down anywhere. The result is a daily decision that takes longer than it should and a wardrobe used at a fraction of its value.

The underlying difficulty is that a wardrobe has no **index**. A person can only see the few garments at the front of the rail, so there is no way to search it, no way to filter it by occasion or season, and no way to record a combination that worked. Existing shopping apps are built to sell new clothes, not to organise clothes already owned.

FitPick gives the wardrobe a digital index. Each garment is recorded once with its category, colour, season, brand, size and price — either by choosing it from a ready-made catalogue or by entering it by hand. From then on the collection can be searched, filtered and sorted in seconds, garments can be combined into named outfits that are saved and scored, and the application reports what is actually owned and how often it is worn.

1.2 Who it is for

Persona	Situation	What FitPick gives them
Priya, 20 College student	A moderate wardrobe on a tight budget. Repeats the same few outfits and has bought duplicates by mistake.	A complete view of what she already owns, so she stops re-buying and starts re-combining.
Arjun, 29 Working professional	Needs formal and smart-casual looks ready for the week and has no time to decide each morning.	Saved outfits tagged by occasion and scored, so getting dressed becomes choosing from a short list.
Meera, 24 Fashion enthusiast	A large wardrobe she enjoys experimenting with, but no record of what she has tried.	An outfit builder, favourites, and statistics about categories, colours and cost per wear.

1.3 Vision and objectives

Vision

To make wardrobe management simple and organised, so that every user can see what they own, plan outfits easily, and get more value from the clothes they already have.

The objectives that follow from this vision are:

1. Let a user record what they own with as little effort as possible — ideally by picking from a list rather than typing.
2. Make any garment findable by name, brand, category or season in a single interaction.
3. Let a user assemble and save an outfit from existing garments, reopen it later, and get useful feedback on it.
4. Report honest statistics about the wardrobe — totals, favourites, most worn, value, wear rate.
5. Keep each person's wardrobe separate from everyone else's.
6. Run identically on any machine so the project can be demonstrated and marked reliably.

1.4 Scope

In scope	Out of scope for this version
Recording, editing and removing clothing items	AI or machine-learning outfit recommendation
Choosing garments from a ready-made catalogue	Weather-based or calendar-based suggestions
Searching, filtering and sorting the wardrobe	Social features — sharing, following, comments
Building, saving, scoring and favouriting outfits	Shopping, price tracking or retailer integration
Separate wardrobes per account	Password authentication and sessions
Dashboard and wardrobe analytics	Photograph upload and storage

In scope	Out of scope for this version
Containerised local deployment	Cloud hosting, CI/CD pipeline, mobile applications

1.5 Constraints and assumptions

- **Constraint —** an academic project built by one developer against a fixed deadline, so the technology stack was kept deliberately small.
- **Constraint —** must run locally with free, open tooling; no paid cloud services.
- **Constraint —** no password authentication in this version. The email given at sign-in identifies an account and keeps wardrobes apart, but it does not authenticate anyone; the `password_hash` column exists unused.
- **Assumption —** recording a wardrobe must be nearly effortless, which is why the catalogue was added: picking from a list beats typing or photographing forty garments.
- **Assumption —** photographs are optional. Text details alone are enough, so the application illustrates each garment from its category and colour.
- **Assumption —** users access FitPick from a laptop or phone browser; no native application is needed.

1.6 Success metrics

Metric	Target
Time to record a first clothing item after opening the app	under 2 minutes
Time to assemble and save an outfit	under 1 minute
Wardrobe search response for a 500-item wardrobe	under 1 second
Test users rating the interface “easy to use” or better	80%

2. User stories

Requirements were captured as user stories, written as GitHub issues and prioritised with **MoSCoW** on a project board. Fifteen stories were placed in **Must Have** and ten in **Should Have**. The Could Have and Won't Have columns were left empty on purpose: anything that did not serve the vision was kept out of the backlog entirely rather than parked in it.

2.1 Must Have

The **Status** column states honestly what exists today. “Full” means the story works end to end through the interface; “API” means the endpoint and its tests exist but no screen calls it yet; “Partial” means the core works but part of the story is missing; “UI only” means the screen exists with no backend behind it.

#	As a user I want to...	...so that	Status
1	register an account	my wardrobe is private to me	Partial
2	log in	I can reach my own wardrobe	Partial
3	log out	I can leave the app safely on a shared computer	Full
4	add a clothing item	my wardrobe is recorded in one place	Full
5	edit a clothing item	I can correct details I entered wrongly	API
6	delete a clothing item	items I no longer own stop cluttering the list	Full
7	view my whole wardrobe	I can see everything I own at once	Full
8	search my clothing	I can find one garment without scrolling	Full
9	filter my clothing	I can narrow the wardrobe to what is relevant	Full
10	create an outfit	I can reuse a combination that works	Full
11	edit an outfit	I can adjust a saved look	Partial
12	delete an outfit	outfits I no longer wear are removed	Full
13	save a favourite outfit	my best looks are easy to find	Full
14	view a dashboard	I get an overview without hunting for it	Full
15	update my profile	my account details stay correct	UI only

2.2 Should Have

#	As a user I want to...	...so that	Status
16	organise clothing into categories	the wardrobe has a structure I recognise	Full
17	tag items and outfits by occasion	I can dress for the day's plan	Full
18	see recently added clothing	I notice what I have just logged	Full
19	see wardrobe statistics	I understand what I actually own	Full
20	upload a photo of an item	I recognise the garment visually	UI only
21	favourite individual clothing items	my go-to pieces are one click away	Full
22	view the details of one item	I can check size, brand and notes	API
23	sort my clothing	I can see newest, most worn or most expensive first	Full
24	view my outfit history	I can return to looks I built before	Full
25	change my password	my account stays secure	UI only

2.3 Delivery summary

Status	Count	Meaning
Full	17	Working end to end from the interface against the API and database
Partial	3	Core behaviour works; one part of the story is still missing

Status	Count	Meaning
API only	2	Endpoint implemented and tested, but no screen calls it yet
UI only	3	Screen exists; the supporting backend is listed as future work
Total	25	15 Must Have and 10 Should Have

2.4 Stories added after the first submission

The instructor asked for changes after the original submission. These are new stories, not part of the original twenty-five, and all four are complete:

#	As a user I want to...	...so that	Status
26	have a wardrobe of my own	what I own is separate from anyone else's	Full
27	pick clothes from a ready-made catalogue	I do not have to type in or photograph every garment	Full
28	remove a piece from my wardrobe	the list reflects what I actually own	Full
29	see each saved outfit scored	I get feedback on the combinations I build	Full

2.5 A story in full

Each story carries acceptance criteria. Story 27 is shown as an example of how the new backlog items were written:

Story 27 — Pick clothes from the catalogue

As a FitPick user **I want to** choose garments from a ready-made catalogue **so that** I do not have to type in or photograph every piece I own.

Acceptance criteria

1. The catalogue offers ready-made pieces across all six categories and can be filtered by category.
2. Browsing the catalogue does not change any wardrobe.
3. Choosing a piece copies it into the signed-in account's wardrobe only.
4. A piece already owned is marked, and may still be added again.
5. Choosing a piece updates the dashboard statistics immediately.
6. An unknown catalogue id is rejected with HTTP 404.

Every one of those six criteria is covered by an automated test in `tests/test_integration_catalogue.py`. The full traceability from stories to tests is in Section 4.5.

3. System architecture and system design

3.1 Architecture style

FitPick uses a **layered (three-tier) architecture**. The **presentation tier** is a React single-page application; the **application tier** is a FastAPI REST service holding all business rules; the **data tier** is PostgreSQL. Calls travel downwards only — the browser talks to the API, the API talks to the database, and no tier skips a level or calls upwards.

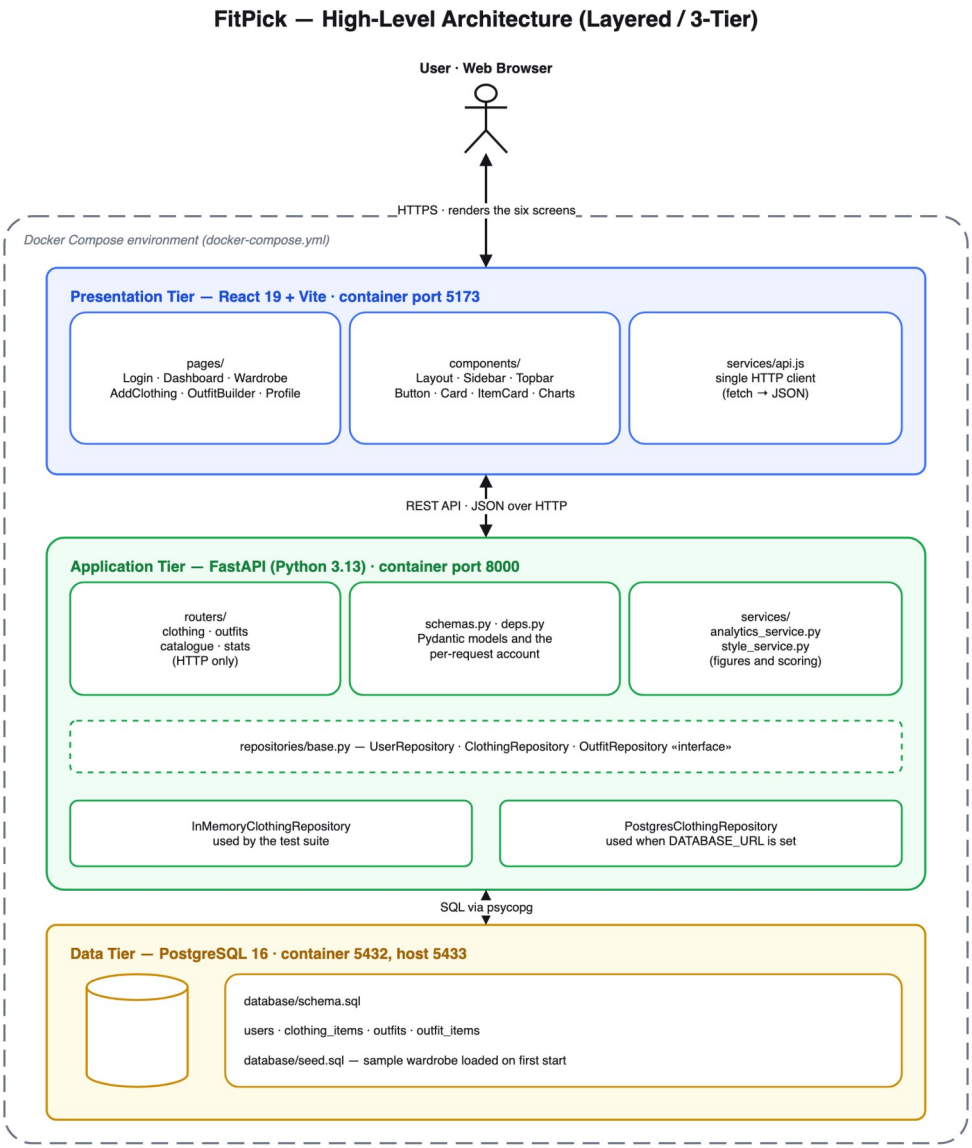


Figure 1 — FitPick high-level architecture (editable source: docs/design/architecture/fitpick-architecture.drawio)

3.2 Why this style was chosen

- **It fits the problem.** FitPick is a CRUD application over a small relational model. A layered design is the standard, proven shape for that, and avoids the operational cost of micro-services for a system with one developer and one database.
- **It enforces separation of concerns.** The UI never writes SQL and the database never knows React exists, so each tier can be understood, tested and replaced on its own.
- **It absorbed the revision without restructuring.** Per-account wardrobes, a catalogue and outfit scoring were each added as one new module inside an existing tier — no layer boundary moved.
- **It is simple to build and explain.** One command starts the whole system, and the single diagram above describes it completely.

3.3 Module responsibilities

Module	Responsibility	Must not
`frontend/src/pages/`	One file per screen; holds screen state and arranges components	know the backend URL or build HTTP requests
`frontend/src/components/`	Reusable presentation — Layout, Sidebar, Topbar, Button, Card, ItemCard, Charts, UserMenu	know which screen is rendering it
`frontend/src/services/`	api.js is the only module that talks to the backend; session.js holds who is signed in	contain screen logic or formatting
`backend/deps.py`	Turn the request's identity headers into the owning user row	contain business rules
`backend/routers/`	Translate HTTP to repository and service calls, and set status codes	contain storage logic or calculations
`backend/schemas.py`	Pydantic models that define the API contract and validate input	know how data is stored
`backend/services/`	Business calculations: dashboard figures and the outfit style score	perform I/O or know about HTTP
`backend/repositories/`	Storage behind one interface, with in-memory and PostgreSQL implementations	know about HTTP or business rules
`backend/catalogue.py`	The read-only list of ready-made garments	know about any particular user

3.4 Design principles in the code

Abstraction

Each layer exposes a small, stable interface and hides how it works. Pages never call `fetch`; they call named functions and receive plain objects. Routers never touch storage; they call a repository. Because the routers depend only on the interface, FitPick runs against an in-memory store or PostgreSQL with no change to any endpoint — one place decides which:

backend/repositories/__init__.py

```
def _build() -> tuple[ClothingRepository, OutfitRepository, UserRepository]:
    dsn = os.environ.get("DATABASE_URL")
    if dsn:
        from repositories.postgres_repository import (
            PostgresClothingRepository,
            PostgresOutfitRepository,
            PostgresUserRepository,
        )
        return (
            PostgresClothingRepository(dsn),
            PostgresOutfitRepository(dsn),
            PostgresUserRepository(dsn),
        )
```

Modularity

The codebase is a set of small files with one job each. The clearest evidence is the revision itself: **per-account wardrobes, the catalogue and outfit scoring were each added as one new module** — `deps.py`, `catalogue.py` with its router, and `services/style_service.py` — plus one line of registration each. No existing module had to be rewritten.

backend/main.py — a new feature is a new router plus one line

```
from routers import catalogue, clothing, outfits, stats

app.include_router(catalogue.router)
app.include_router(clothing.router)
app.include_router(outfits.router)
app.include_router(stats.router)
```

Cohesion

`Sidebar.jsx` only renders navigation. `style_service.py` only scores an outfit and performs no I/O. `deps.py` only resolves identity. `catalogue.py` only holds the garment list. Most files are under eighty lines, and each can be read

and tested on its own — which is precisely why the style service could be unit-tested thirty-two times without a database or an HTTP client.

Low coupling

Modules communicate only through explicit interfaces: React props, the single `api.js` client, the Pydantic schemas, and the repository interface. The identity of the signed-in user reaches the backend as two HTTP headers and is converted to a user id in exactly one place, so no router contains identity logic:

backend/deps.py

```
def current_user_id(
    x_user_email: str | None = Header(default=None),
    x_user_name: str | None = Header(default=None),
    users: UserRepository = Depends(get_user_repository),
) -> int:
    email = (x_user_email or DEMO_EMAIL).strip().lower()
    name = unquote(x_user_name or DEMO_NAME).strip() or email
    return users.get_or_create(email=email, name=name).id
```

3.5 User interface design

The screens were designed in Figma and implemented as React pages. The brand mark returns to the Dashboard from anywhere, and the account chip opens a menu holding profile and log out. They share one design system: royal blue #3B6FF6 for primary actions and the active navigation item, white cards with rounded corners and soft shadows on a light grey #F6F7F9 ground, and a single sans-serif typeface.

Decision	Effect on usability
One fixed sidebar on every screen after login	Navigation never moves, so the layout is learned once
Exactly one blue primary button per screen	The next step is always obvious; everything else is secondary
A catalogue instead of a blank form for a new user	Removes the biggest barrier to adoption — the initial data entry
An empty wardrobe states what to do next, rather than showing nothing	A new account is never a dead end
Removing a piece confirms in place	One click is never destructive
Each saved outfit shows why it scored what it did	The score can be argued with rather than simply accepted
Garments drawn from category and colour when no photo exists	The wardrobe looks complete from the first item

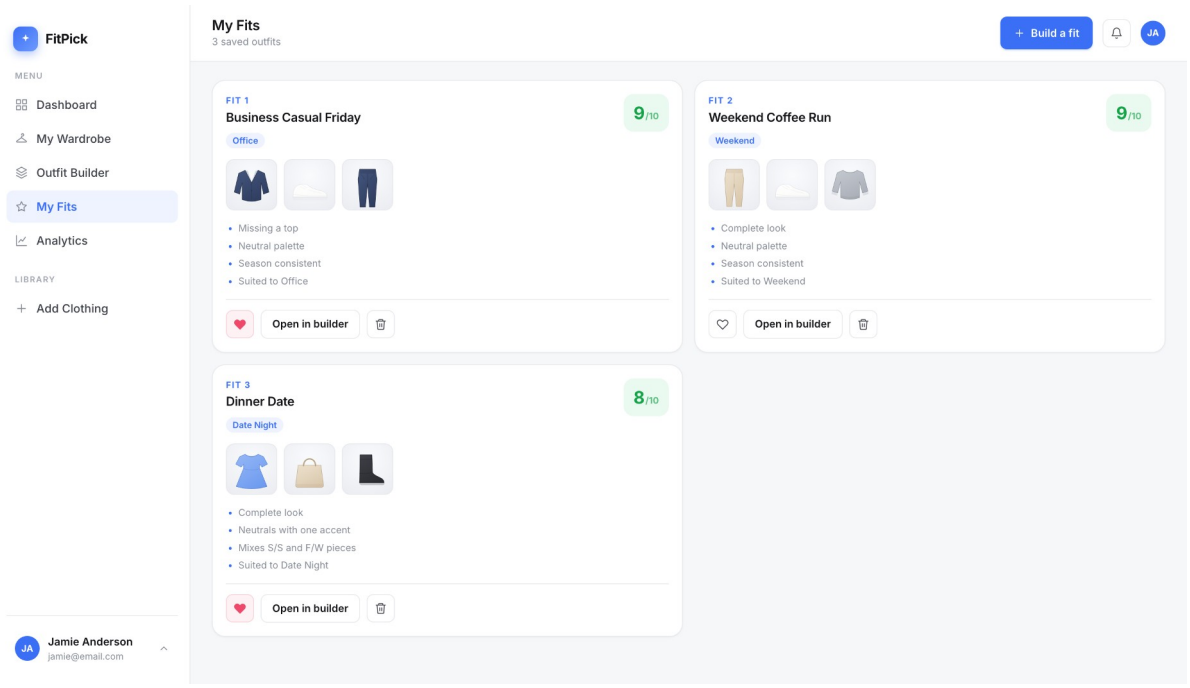


Figure 2 — My Fits: saved outfits numbered and scored, each with its reasons. The other screens are in Part A, Section 8.

3.6 Key design decisions

Decision	Alternative considered	Why this one
Layered 3-tier architecture	Micro-services	One developer, one database and a CRUD workload; micro-services would add deployment cost and no benefit
Repository interface with two implementations	Calling SQL directly from routers	The 96 tests run with no database, and the storage engine can change without touching an endpoint
Identity as a request header resolved in one dependency	Passing a user id through every function by hand	Keeps routers free of identity logic and makes the demo account a one-line fallback
Separate services for figures and scoring	Calculations inside routers	Keeps both testable without HTTP and keeps routers cohesive
A shared read-only catalogue	Seeding every new account with sample clothes	A new wardrobe is genuinely the user's own, and the catalogue stays one definition for everyone
A rule-based outfit score	A random number, or a learned model	Deterministic, explainable and unit-testable; a random score could not be tested or defended, and a learned model is out of scope
Normalised PostgreSQL schema with a join table	Storing outfit items as a JSON array	An outfit-to-item link is genuinely many-to-many; foreign keys keep it consistent
SVG garments generated from category and colour	Bundled stock photographs	No licensing or storage concern, consistent look, and real photo upload can replace it later

4. Design of tests

Tests were designed from the acceptance criteria of the user stories, not written afterwards to match the code. Three levels are used, and a fourth technique — mutation testing — measures whether those tests are actually strong. The full run output is in Part A; this section explains how the cases were **designed**.

4.1 Test levels and their purpose

Level	What it proves	Tests
Unit	One calculation is correct in isolation, including its boundaries	43
Integration	Identity, routing, validation, serialisation and storage work together over real HTTP	43
Regression	A fault that was found once cannot return	10
Mutation	The tests above would actually fail if the code were wrong	131 mutants

4.2 Test-case design techniques

Cases were derived using standard black-box techniques rather than chosen arbitrarily:

Technique	Applied to	Cases produced
Equivalence partitioning	The category field: valid categories vs. anything else	One valid category is accepted; “Hats” is rejected with HTTP 422
Boundary value analysis	“Added this week”, which uses a seven-day cut-off	Items 0, 7 and 8 days old; only the first two count
Boundary value analysis	The accent-colour thresholds in the outfit score	Zero, one, two and three accents, each asserting an exact score
State transition testing	The favourite flag on an item	Off → on → off must return to the original value
Error guessing	A brand-new account with nothing in it	Empty wardrobe must not divide by zero or call max() on an empty list
Access control	Two accounts and one item	Reading, deleting and counting another account's item must all fail
Interface testing	Every REST endpoint	Correct status codes: 200, 201, 204, 404, 422

4.3 Representative test cases

ID	Pre-condition	Action	Expected result
TC-01	Seeded demo wardrobe of 12 items	POST /clothing with a valid body	201 Created; the item is returned with a new id and appears in the list
TC-02	Seeded wardrobe	POST /clothing with category “Hats”	422 Unprocessable Entity; nothing is stored
TC-03	Seeded wardrobe	GET /clothing?search=levi	Only “Dark Wash Jeans” is returned (brand match)
TC-04	A new email never seen before	GET /clothing with that identity	An empty list; the demo wardrobe is untouched
TC-05	Account A owns one item	GET /clothing/{id} as account B	404 Not Found
TC-06	A new account	GET /catalogue then GET /clothing	The catalogue lists 33 pieces; the wardrobe is still empty
TC-07	A new account	POST /catalogue/add with a valid id	201 Created; that piece, and only that piece, is in the wardrobe
TC-08	Complete neutral in-season outfit tagged for the occasion	score_outfit(items, occasion)	10 of 10, with “Complete look” as the first reason
TC-09	Outfit mixing an S/S and an F/W piece	score_outfit(items, occasion)	Season points lost; reason reads “Mixes S/S and F/W pieces”
TC-10	Empty wardrobe	analytics_stats(repository)	average_wears is 0.0 and top_category is the placeholder, with no exception

4.4 Test data strategy

A twelve-item seed wardrobe is defined once in `backend/seed_data.py` and mirrored in `database/seed.sql`, and it belongs to the demo account. Tests that need a different shape of data build their own: the `empty_wardrobe`

fixture supplies an unseeded repository pair, and the account tests simply send a new email, which is itself the behaviour under test.

backend/tests/test_style_unit.py — a helper builds precise data for boundary cases

```
def piece(category, colour="Black", season="All Season", tags=None, name=None):
    return ClothingItem(
        id=0,
        name=name or f"{colour} {category}",
        category=category,
        colour=colour,
        season=season,
        tags=tags or [],
    )


FULL = [
```

4.5 Traceability: user story to test

Every story with implemented behaviour is covered by at least one automated test:

Story	Covering tests
4 Add clothing item	test_add_item_then_read_it_back, test_add_item_increases_wardrobe_count, test_rejects_unknown_category, test_rejects_item_without_a_name
5 Edit clothing item	test_update_item_changes_only_given_fields, test_reg07_updating_one_field_keeps_the_others
6 Delete clothing item	test_delete_item_removes_it, test_reg02_deleting_a_missing_item_is_a_404, test_one_account_cannot_delete_another_accounts_item
7 View wardrobe	test_seeded_wardrobe_is_returned
8 Search clothing	test_search_matches_item_name, test_reg01_search_also_matches_the_brand, test_reg08_search_is_case_insensitive
9 Filter clothing	test_filter_by_category, test_reg03_category_all_does_not_filter_anything_out
10 Create outfit	test_create_outfit_from_existing_items, test_outfit_needs_at_least_one_item, test_outfit_rejects_unknown_item
12 Delete outfit	test_delete_outfit
13 Favourite outfit	test_favourite_an_outfit
14 View dashboard	test_dashboard_totals_match_the_wardrobe, test_breakdown_adds_up_to_the_total, test_reg10_deleted_item_disappears_from_the_statistics
19 Wardrobe statistics	test_analytics_wardrobe_value_is_the_sum_of_prices and 10 further unit tests
21 Favourite clothing items	test_toggle_favourite_flips_the_flag, test_reg06_favourite_toggles_back_to_its_original_value
23 Sort clothing items	test_sort_by_most_worn
26 A wardrobe of my own	All 9 tests in test_integration_accounts.py
27 Pick from the catalogue	All 9 tests in test_integration_catalogue.py
28 Remove a piece	test_delete_item_removes_it, test_one_account_cannot_delete_another_accounts_item
29 See each outfit scored	All 32 tests in test_style_unit.py

4.6 Results

Test outcome

96 of 96 tests pass.
Mutation score over both service modules 98.5% (129 of 131), improved from 79.4% after twenty mutation-driven unit tests. Of the two survivors, one is caught by the suite but misreported by the tool and one is provably equivalent.
Frontend: ESLint clean and the production build succeeds.
Stories 15, 20 and 25 are not covered by automated tests because their backend does not exist yet; they are listed as future work rather than claimed as complete.

5. Appendix A — Software Requirements Specification

A.1 Purpose

This appendix specifies what the FitPick software must do. It is written for the developer and the assessor, and is the reference against which the tests in Section 4 were designed.

A.2 Product perspective

FitPick is a self-contained web application with no dependency on an external system. It consists of a browser client, a REST API and a relational database. In development these are three containers; in a deployed build the compiled client is served by the API container, so the site and its API share one origin.

A.3 Functional requirements

ID	Requirement	Priority	Status
FR-01	The system shall allow a user to record a clothing item with name, category, subcategory, colour, season, brand, size, price, tags and notes.	Must	Done
FR-02	The system shall reject a clothing item whose category is not one of the six defined categories.	Must	Done
FR-03	The system shall reject a clothing item with no name or no colour.	Must	Done
FR-04	The system shall list all clothing items belonging to the signed-in account.	Must	Done
FR-05	The system shall filter clothing items by free-text search over name and brand.	Must	Done
FR-06	The system shall filter clothing items by category and by season.	Must	Done
FR-07	The system shall sort clothing items by newest, name, most worn or price.	Should	Done
FR-08	The system shall update any subset of a clothing item's fields, leaving the others unchanged.	Must	Done
FR-09	The system shall remove a clothing item after an explicit confirmation, and return 404 if it does not exist.	Must	Done
FR-10	The system shall toggle the favourite flag on a clothing item.	Should	Done
FR-11	The system shall create an outfit from one or more existing clothing items.	Must	Done
FR-12	The system shall reject an outfit with no items or with an unknown item id.	Must	Done
FR-13	The system shall list saved outfits, numbered, and allow one to be reopened for editing.	Must	Done
FR-14	The system shall delete an outfit and toggle its favourite flag.	Must	Done
FR-15	The system shall report total items, total outfits, favourites, most-worn item, items added in the last seven days and a per-category breakdown.	Must	Done
FR-16	The system shall report wardrobe value, average wears per item, top category, favourite brand and a colour breakdown.	Should	Done
FR-17	The system shall keep each account's clothing items and outfits separate, and shall not disclose one account's data to another.	Must	Done
FR-18	The system shall create an account on first sign-in and give it an empty wardrobe.	Must	Done
FR-19	The system shall offer a read-only catalogue of predefined garments, filterable by category.	Should	Done
FR-20	The system shall copy a chosen catalogue piece into the signed-in account's wardrobe, and shall reject an unknown catalogue id.	Should	Done
FR-21	The system shall score each saved outfit out of ten by a deterministic rule set and show the reason for each part of the score.	Should	Done
FR-22	The system shall authenticate a user with a stored password hash.	Must	Future
FR-23	The system shall allow a user to update profile details and change password.	Must	Future
FR-24	The system shall store an uploaded photograph for a clothing item.	Should	Future

A.4 Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	A wardrobe query shall return in under one second for a wardrobe of 500 items.
NFR-02	Usability	A new user shall be able to record their first item within two minutes.

ID	Category	Requirement
NFR-03	Portability	The system shall build and run on any machine with Docker, using one command.
NFR-04	Maintainability	No module shall depend on the internals of another; all communication is through defined interfaces.
NFR-05	Testability	The automated test suite shall run without a database and complete in under five seconds.
NFR-06	Reliability	The system shall behave correctly for an empty wardrobe and for missing records.
NFR-07	Security	One account shall not be able to read or modify another account's data. Passwords, when implemented, shall be stored only as hashes.
NFR-08	Compatibility	The interface shall work in current versions of Chrome, Safari, Firefox and Edge.

A.5 External interfaces

Interface	Description
User interface	Seven browser screens, described in Section 3.5 and shown in Part A Section 8
API interface	A JSON REST API; the OpenAPI description is generated at /docs
Identity	X-User-Email and X-User-Name request headers identify the account. This is identification, not authentication.
Database interface	PostgreSQL 16 on container port 5432, published on host port 5433, reached with psycopg
Configuration	DATABASE_URL selects PostgreSQL storage; PORT sets the listening port; VITE_API_URL overrides the backend address

A.6 API specification

Method and path	Purpose	Success
`GET /health`	Liveness probe	200
`GET /catalogue`	The predefined pieces, optionally filtered by category	200
`POST /catalogue/add`	Copy one catalogue piece into the caller's wardrobe	201 / 404
`GET /clothing`	List items, with search, category, season and sort parameters	200
`GET /clothing/categories`	The six valid categories	200
`GET /clothing/{id}`	One item	200 / 404
`POST /clothing`	Create an item	201 / 422
`PUT /clothing/{id}`	Partially update an item	200 / 404
`POST /clothing/{id}/favourite`	Toggle the favourite flag	200 / 404
`DELETE /clothing/{id}`	Remove an item	204 / 404
`GET /outfits`	List saved outfits, each with its style score	200
`GET /outfits/occasions`	The six valid occasions	200
`POST /outfits`	Create an outfit	201 / 422
`POST /outfits/{id}/favourite`	Toggle the favourite flag	200 / 404
`DELETE /outfits/{id}`	Delete an outfit	204 / 404
`GET /stats/dashboard`	Dashboard figures	200
`GET /stats/analytics`	Analytics figures	200

6. Appendix B — Data Flow Diagrams

The data flow diagrams describe FitPick in terms of the data that moves through it. The **context diagram** shows the system as a single process with its one external entity; the **level 1 diagram** decomposes that process into the activities the system performs and the stores it keeps.

B.1 Context diagram (Level 0)

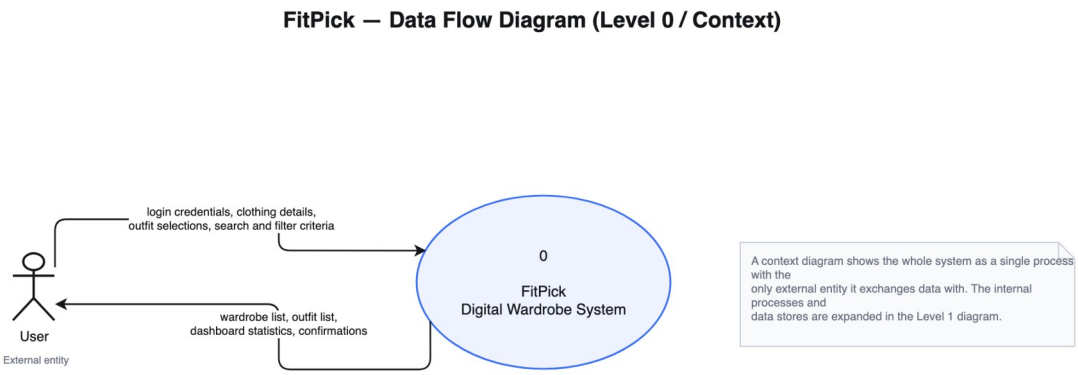


Figure 3 — Level 0 context diagram

B.2 Level 1 diagram

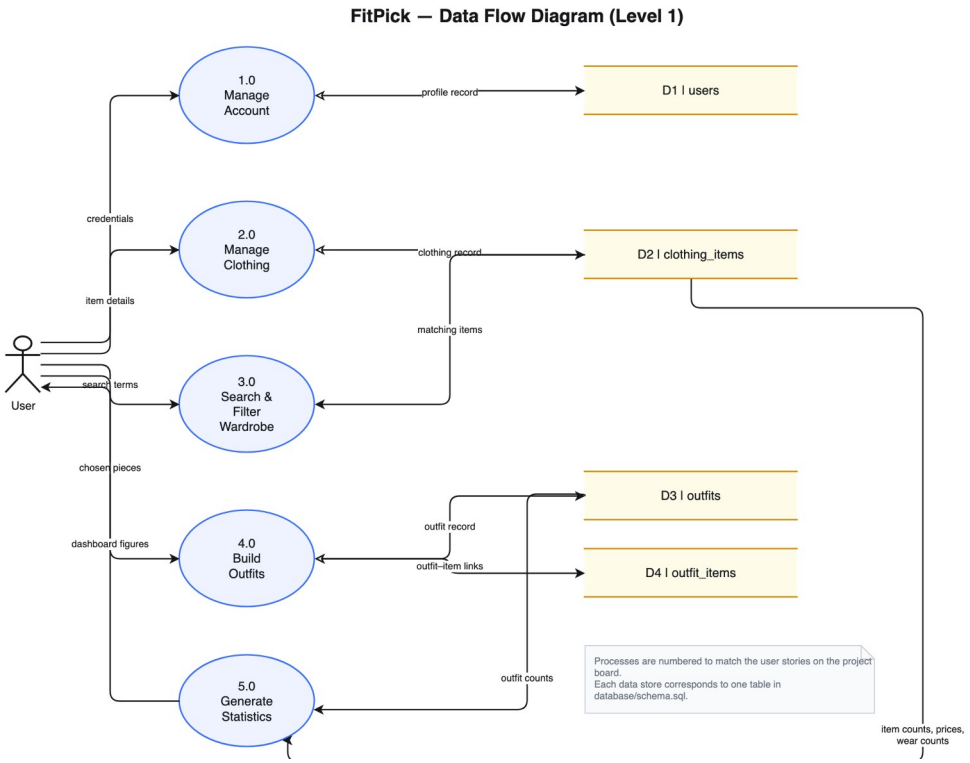


Figure 4 — Level 1 data flow diagram. Each data store maps to one table in database/schema.sql.

B.3 Process descriptions

Process	Input	Processing	Output
1.0 Manage Account	Name and email at sign-in	Find the account for that email, creating it on first use	The owning user id; a row in D1
2.0 Manage Clothing	Item details, item id, catalogue choice	Validate against the schema, then create, update or remove, always scoped to the owner	Clothing record in D2; confirmation
3.0 Search & Filter Wardrobe	Search text, category, season, sort order	Read the owner's rows from D2 and apply the filters and ordering in turn	The matching list of items
4.0 Build Outfits	Chosen item ids, name, occasion	Check every item belongs to the owner, write the outfit and its links, then score it	Outfit in D3; links in D4; a score out of ten
5.0 Generate Statistics	The owner's item and outfit records	Count, sum, average and rank	Dashboard and analytics figures

7. Appendix C — Entity Relationship Diagram

FitPick stores four entities. A user owns many clothing items and creates many outfits; an outfit contains many clothing items and a clothing item can appear in many outfits, so that relationship is resolved by the `outfit_items` join table. The schema is unchanged from the first submission — the revision made the application use the `user_id` foreign key that was already there.

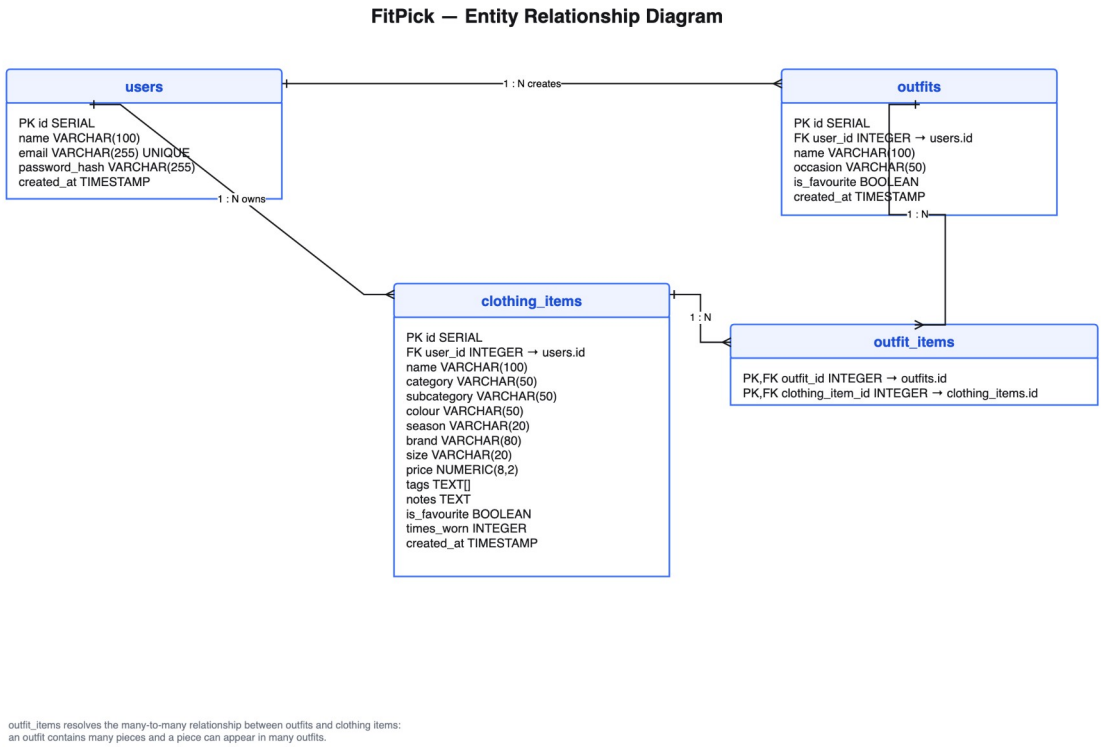


Figure 5 — Entity relationship diagram (crow's-foot notation)

C.1 Data dictionary — clothing_items

Column	Type	Constraint	Meaning
`id`	SERIAL	Primary key	Surrogate identifier
`user_id`	INTEGER	FK → users.id, NOT NULL	Owner of the garment; every query is scoped by it
`name`	VARCHAR(100)	NOT NULL	Display name, e.g. “Navy Blazer”
`category`	VARCHAR(50)	NOT NULL	One of six: Tops, Bottoms, Outerwear, Shoes, Dresses, Accessories
`subcategory`	VARCHAR(50)	Default ''	One of 46 garment types; selects the illustration
`colour`	VARCHAR(50)	NOT NULL	Primary colour; tints the illustration and feeds the style score
`season`	VARCHAR(20)	Default 'All Season'	S/S, F/W or All Season; feeds the style score
`brand`	VARCHAR(80)	Default ''	Used by search and by the favourite-brand statistic
`size`	VARCHAR(20)	Default ''	Free text, because sizing differs by brand
`price`	NUMERIC(8,2)	Default 0	Price paid; feeds the wardrobe-value statistic
`tags`	TEXT[]	Default '{}'	Occasion tags; feed the style score
`notes`	TEXT	Default ''	Free text, e.g. care instructions
`image_url`	TEXT	Nullable	Reserved for the photo-upload story
`is_favourite`	BOOLEAN	Default FALSE	Marked by the heart control in the wardrobe grid
`times_worn`	INTEGER	Default 0	Wear counter; feeds most-worn and average-wears
`created_at`	TIMESTAMP	Default NOW()	Used by “recently added” and “added this week”

C.2 Other tables

Table	Columns	Notes
users	id, name, email (unique), password_hash, created_at	Created on first sign-in. The password hash is reserved; nothing is written to it yet.
outfits	id, user_id, name, occasion, is_favourite, created_at	The style score is derived on read, not stored, so it can never go stale
outfit_items	outfit_id, clothing_item_id	Composite primary key; both columns cascade on delete

C.3 Referential integrity

Every foreign key uses `ON DELETE CASCADE`, so deleting a user removes their wardrobe and outfits, and removing a garment removes it from every outfit that referenced it. This prevents orphan rows without any application-level clean-up code — which is exactly why removing a piece from the wardrobe needed no extra work when that feature was added.

database/schema.sql — the join table that resolves the many-to-many relationship

```
-- Join table: which clothing items make up an outfit (many-to-many)
CREATE TABLE IF NOT EXISTS outfit_items (
    outfit_id        INTEGER NOT NULL REFERENCES outfits(id) ON DELETE CASCADE,
    clothing_item_id INTEGER NOT NULL REFERENCES clothing_items(id) ON DELETE CASCADE,
    PRIMARY KEY (outfit_id, clothing_item_id)
);

CREATE INDEX IF NOT EXISTS idx_clothing_user     ON clothing_items (user_id);
CREATE INDEX IF NOT EXISTS idx_clothing_category ON clothing_items (category);
CREATE INDEX IF NOT EXISTS idx_outfits_user      ON outfits (user_id);
```

8. Appendix D — UML diagrams

D.1 Use case diagram

Two actors interact with FitPick. A **Guest** can only register or log in. A **Registered User** reaches everything else. The grouped use cases on the left are the ones a user starts directly; each includes the detailed sub-functions on the right.

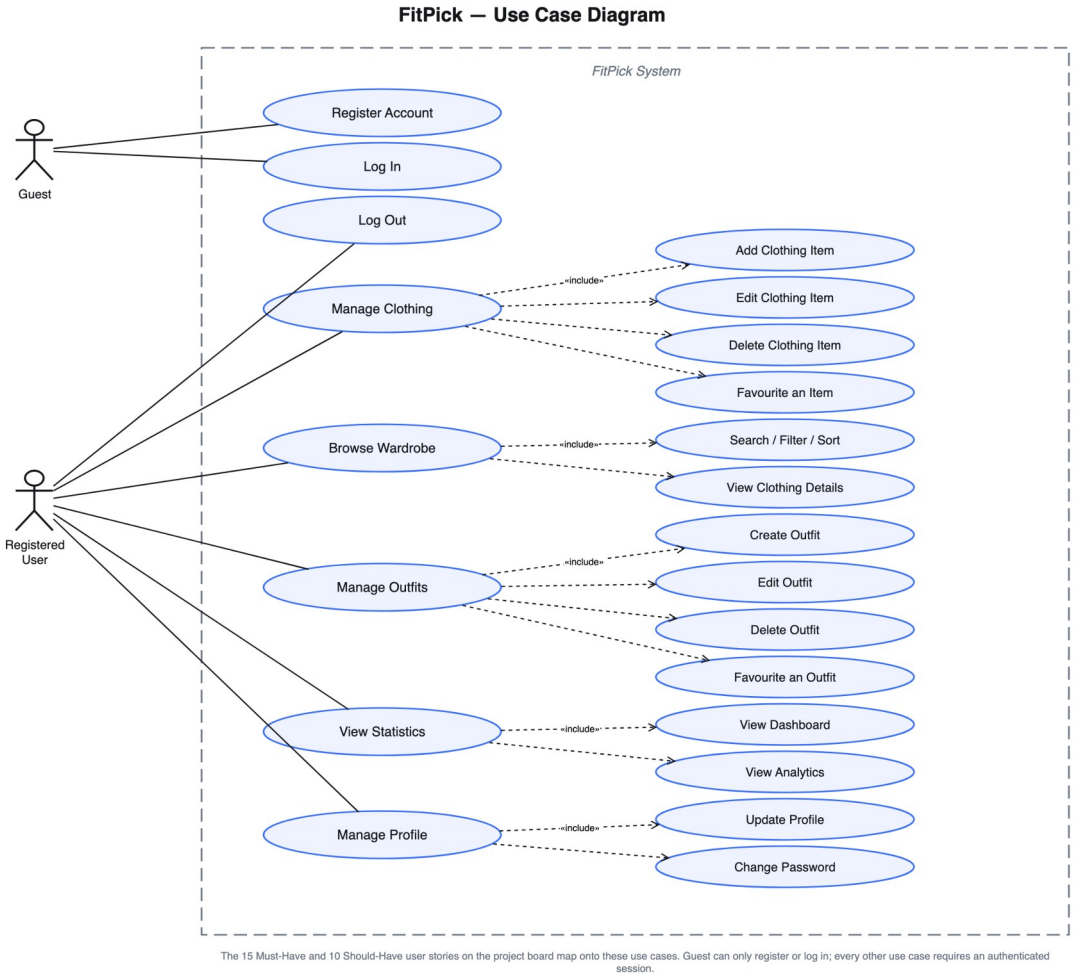


Figure 6 — Use case diagram

D.2 Class diagram

The backend class diagram shows the structure that makes the storage layer replaceable. `ClothingRepository` is an abstract interface; two concrete classes implement it, and the routers and both services depend on the interface alone. `StyleService` was added in this revision and reads nothing but the outfit's own pieces.

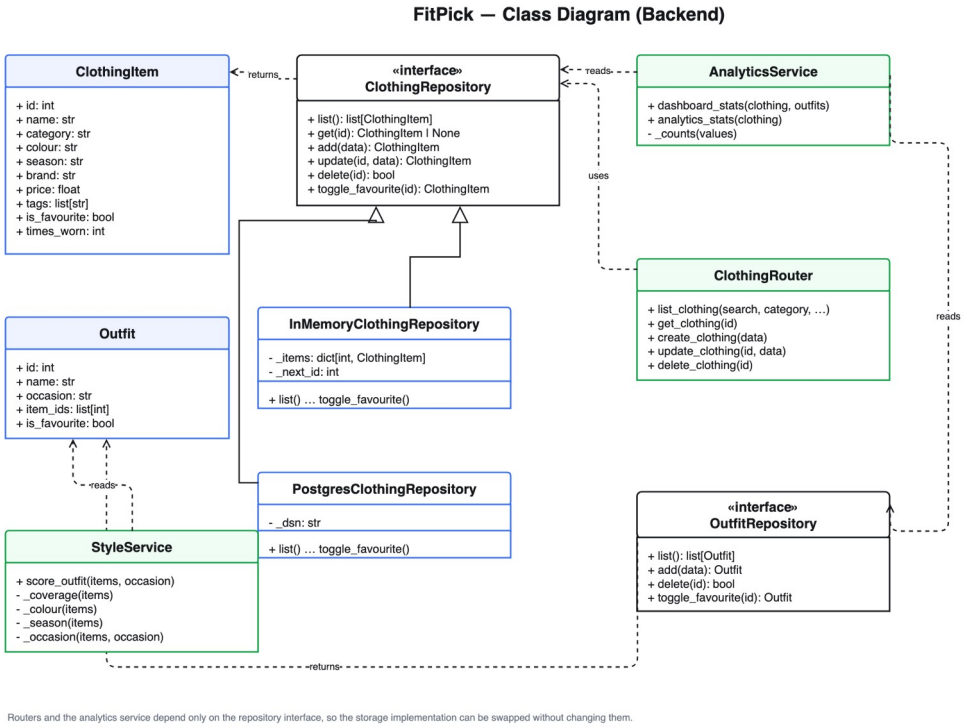


Figure 7 — Class diagram of the backend

D.3 Sequence diagram

Adding a clothing item is the fullest path through the system — it touches every tier and has both a success and a failure branch:

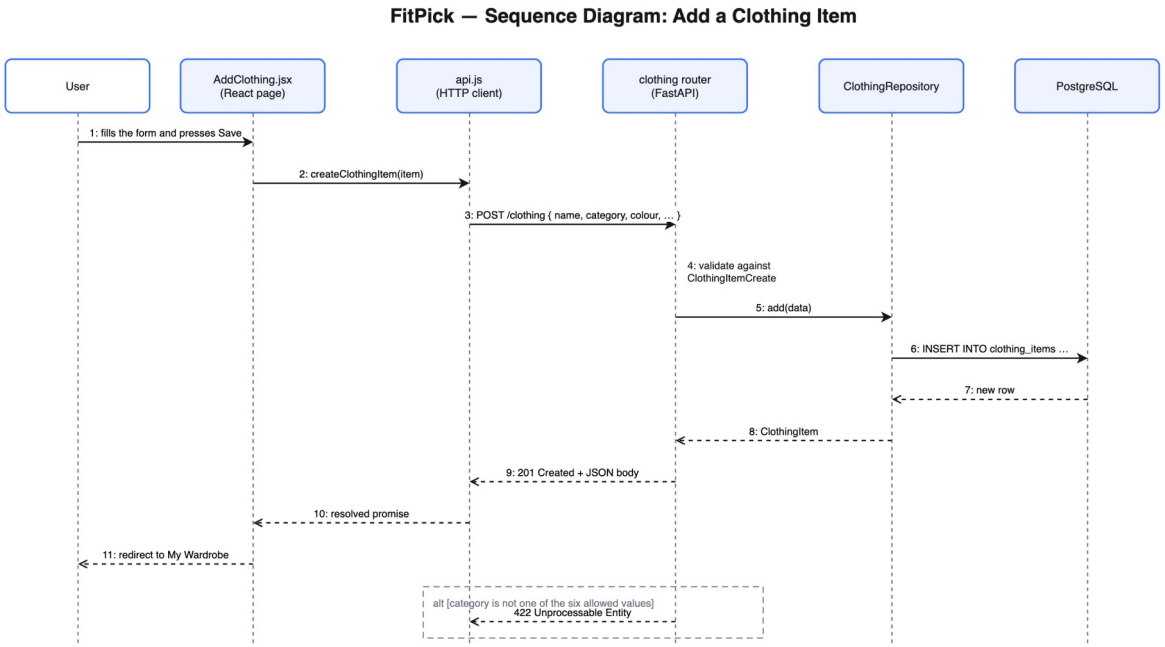


Figure 8 — Sequence diagram: adding a clothing item

9. Appendix E — Code listing and GitHub link

E.1 Repository

Source code

https://github.com/nithi0307/fitpick

The repository contains the full source, the 96 automated tests, the editable Draw.io diagram sources, the exported PNGs, every screenshot used in these reports, a demo script, and both report documents.

E.2 Repository structure

```
fitpick/
├── frontend/src/
│   ├── App.jsx                    route table
│   ├── pages/                     Login, Dashboard, Wardrobe, AddClothing,
│   │                              OutfitBuilder, MyFits, Profile
│   ├── components/                Layout, Sidebar, Topbar, UserMenu, Button, Card,
│   │                              ItemCard, CataloguePicker, Charts, Garment, Icon
│   ├── services/api.js            the only module that calls the backend
│   ├── services/session.js        who is signed in
│   └── styles/global.css          design tokens and shared component styles
├── backend/
│   ├── main.py                    app factory, CORS, routers, static serving
│   ├── deps.py                    resolves the request's account
│   ├── catalogue.py               the 33 predefined garments
│   ├── routers/                   clothing · outfits · catalogue · stats
│   ├── schemas.py                 Pydantic models — the API contract
│   ├── services/                  analytics_service.py · style_service.py
│   ├── repositories/              base.py, memory_repository.py,
│   │                              postgres_repository.py, __init__.py (factory)
│   ├── seed_data.py               the demo account's sample wardrobe
│   └── tests/                     96 tests: unit, integration, regression
├── database/schema.sql, seed.sql
├── docs/
│   ├── design/architecture/       architecture .drawio + .png
│   ├── design/diagrams/           DFD, ERD, use case, class, sequence
│   ├── screenshots/               application and terminal evidence
│   ├── DEMO-SCRIPT.md             a seven-minute walkthrough
│   └── *.pdf                      this report and the testing report
├── Dockerfile                     production image (app + API, one origin)
├── docker-compose.yml             development: three containers
├── render.yaml                    deployment blueprint
└── README.md
```

E.3 Key listings

E.3.1 Storage interface — backend/repositories/base.py

```
class UserRepository(ABC):
    @abstractmethod
    def get_or_create(self, email: str, name: str) -> User: ...


class ClothingRepository(ABC):
    @abstractmethod
    def list(self, user_id: int) -> list[ClothingItem]: ...

    @abstractmethod
    def get(self, user_id: int, item_id: int) -> ClothingItem | None: ...

    @abstractmethod
    def add(self, user_id: int, data: ClothingItemCreate) -> ClothingItem: ...

    @abstractmethod
    def update(
        self, user_id: int, item_id: int, data: ClothingItemUpdate
    ) -> ClothingItem | None: ...

    @abstractmethod
```

E.3.2 Identity for a request — backend/deps.py

```
from urllib.parse import unquote

from fastapi import Depends, Header

from repositories import get_user_repository
from repositories.base import UserRepository

DEMO_EMAIL = "jamie@email.com"
DEMO_NAME = "Jamie Anderson"



def current_user_id(
    x_user_email: str | None = Header(default=None),
    x_user_name: str | None = Header(default=None),
    users: UserRepository = Depends(get_user_repository),
) -> int:
    email = (x_user_email or DEMO_EMAIL).strip().lower()
    name = unquote(x_user_name or DEMO_NAME).strip() or email
    return users.get_or_create(email=email, name=name).id
```

E.3.3 The outfit style score — backend/services/style_service.py

```
MAX_COVERAGE, MAX_COLOUR, MAX_SEASON, MAX_OCCASION = 4, 3, 2, 1
MAX_SCORE = MAX_COVERAGE + MAX_COLOUR + MAX_SEASON + MAX_OCCASION

def _colour(items: list[ClothingItem]) -> tuple[int, str]:
    """Neutrals always sit together; one accent lifts them; three fight."""
    accents = {
        item.colour.strip().lower()
        for item in items
        if item.colour.strip().lower() not in NEUTRALS
    }

    if len(accents) <= 1:
        note = "Neutral palette" if not accents else f"Neutrals with one accent"
        return MAX_COLOUR, note
    if len(accents) == 2:
        return 2, "Two accent colours"
    return 1, f"{len(accents)} accent colours competing"



def score_outfit(items: list[ClothingItem], occasion: str) -> tuple[int, list[str]]:
    """Return the score out of ten and the reason for each part of it."""
    if not items:
        return 0, ["No pieces chosen"]

    results = [
        _coverage(items),
        _colour(items),
        _season(items),
        _occasion(items, occasion),
    ]
    total = sum(points for points, _ in results)
    return total, [note for _, note in results]
```

E.3.4 The catalogue router — backend/routers/catalogue.py

```


@router.get("", response_model=list[CatalogueItem])
def list_catalogue(category: str = "All"):
    entries = catalogue_data.entries()
    if category != "All":
        entries = [e for e in entries if e["category"] == category]
    return entries



@router.post("/add", response_model=ClothingItem, status_code=201)
def add_from_catalogue(
    data: AddFromCatalogue,
    repository: ClothingRepository = Depends(get_clothing_repository),
    user_id: int = Depends(current_user_id),
):
    entry = catalogue_data.find(data.catalogue_id)
    if entry is None:
        raise HTTPException(status_code=404, detail="Catalogue item not found")

    fields = {key: entry[key] for key in catalogue_data.FIELDS}
    return repository.add(user_id, ClothingItemCreate(**fields))
```

E.3.5 Who is signed in — frontend/src/services/session.js

```
  if (words.length === 0) return email;
  return words.map((w) => w[0].toUpperCase() + w.slice(1)).join(" ");
}

export function signIn({ name, email }) {
  const address = (email ?? "").trim().toLowerCase();
  const display = (name ?? "").trim() || nameFromEmail(address);
  const user = { name: display, email: address, initials: initialsFrom(display) };

  try {
    localStorage.setItem(KEY, JSON.stringify(user));
  } catch {
    // private browsing: the session simply lasts for this page view
  }
  return user;
}

export function getSession() {
  try {
    const raw = localStorage.getItem(KEY);
    const user = raw ? JSON.parse(raw) : null;
    return user && user.email ? user : null;
```

E.3.6 Route table — frontend/src/App.jsx

```
 * Route table. Login stands alone; every other screen is nested inside
 * Layout and therefore gets the sidebar automatically.
 */
export default function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Login />} />

        <Route element={<Layout />}>
          <Route path="/dashboard" element={<Dashboard />} />
          <Route path="/wardrobe" element={<Wardrobe />} />
          <Route path="/add-clothing" element={<AddClothing />} />
          <Route path="/outfit-builder" element={<OutfitBuilder />} />
          <Route path="/fits" element={<MyFits />} />
          <Route path="/profile" element={<Profile />} />
        </Route>
      </Routes>
    </BrowserRouter>
  );
}
```

E.4 Running the project

With Docker — the whole system

```
$ git clone https://github.com/nithi0307/fitpick.git
$ cd fitpick
$ docker compose up --build
```

Running the tests

```
$ cd backend && python3.13 -m venv .venv
$ .venv/bin/pip install -r requirements-dev.txt
$ .venv/bin/python -m pytest        # 96 tests
$ .venv/bin/mutmut run              # mutation testing over services/
```

E.5 Future work

- Real authentication: password hashing and sessions, replacing the header-based identity. The users table and its password_hash column are already in place.
- Screens for editing a clothing item and viewing one in detail, so stories 5 and 22 become end-to-end rather than API-only.
- Photograph upload and storage, replacing the generated garment illustrations.
- A wear-logging action so times_worn is updated by the user rather than seeded.
- Extending the style score with wardrobe history — for example rewarding an outfit that uses a piece which has not been worn recently.